

A Summary of my PhD Thesis: *Mathematical Modelling of Hybrid Photonic Structures for Holographic Sensors*

Jack Lyons, PhD

Motivation & Research Questions

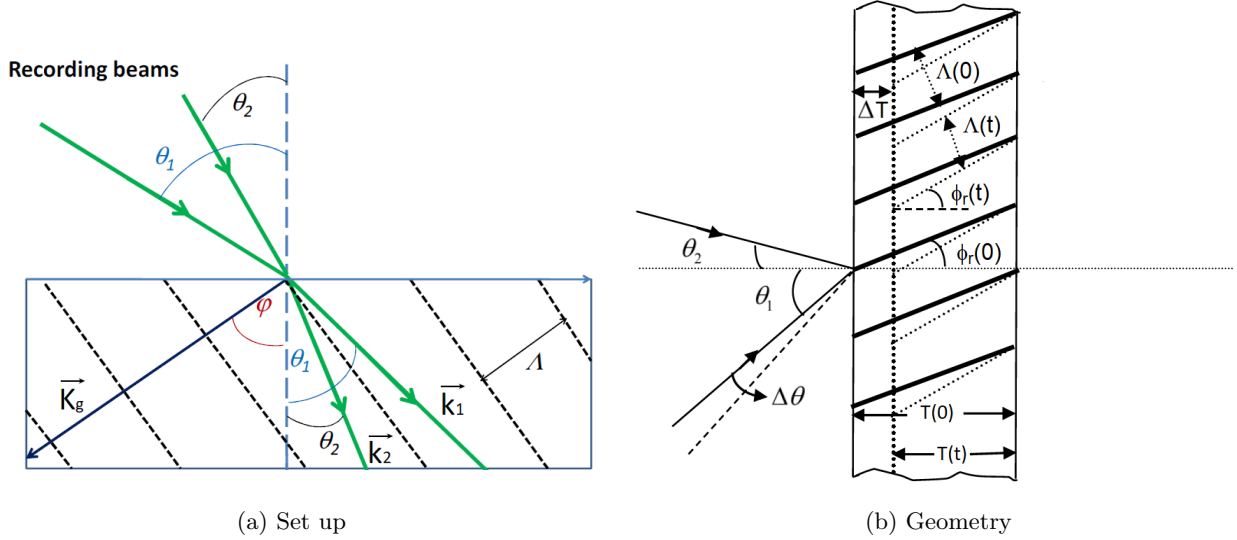


Figure 1: The optical set up and geometry for a holographic grating.

The motivation for this research project was as follows:

- Optical properties of holographic gratings are sensitive to external stimulus and can be exploited for the purpose of environmental sensing.
- Hybrid photopolymers are a strong candidate as recording media for holographic gratings because they offer a wide dynamic range and good selectivity.
- A mathematical framework modelling the formation and operation of a holographic sensor is needed to optimize the design.

The successful completion of this research was characterized by the ability to answer the following research questions.

1. How can we use the ...

- (a) host photopolymer material properties (monomer, binder matrix, dye, etc.)
- (b) recording conditions (recording intensity, spatial frequency, etc.)
- (c) nanoparticle properties (initial concentration, refractive index, etc.)

in order to control the final grating in a hybrid polymer system and hence optimize the functionality of a holographic sensor?

2. Can the theoretical model predict IEO experimental results:

- (a) Significantly increased dynamic range over conventional photopolymer media.

- (b) Non-linear response of refractive index modulation to increased doping.
- (c) Photopolymerization-induced shrinkage is significantly reduced by the addition of zeolite nanoparticles.
- (d) Increased shrinkage at high spatial frequencies.

Mathematical Model

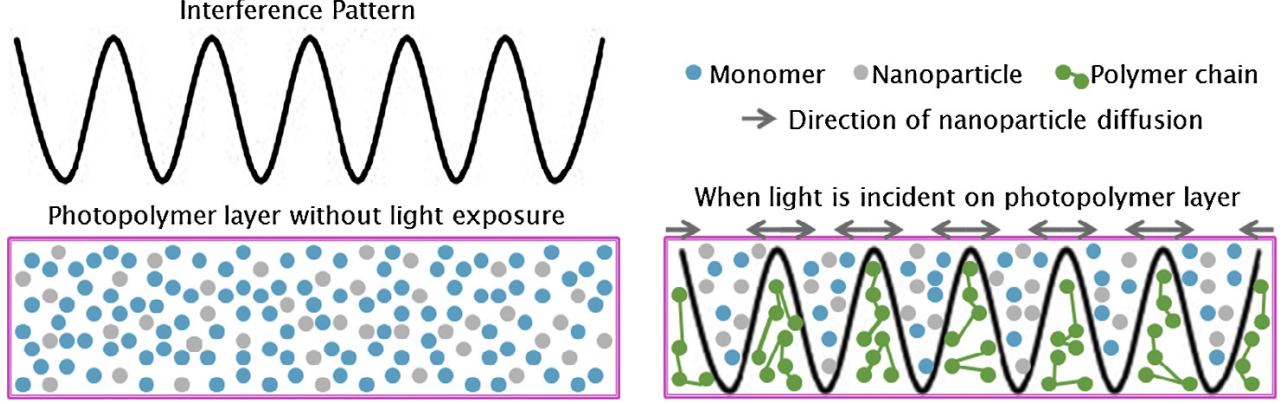


Figure 2

In blue, the previous model and in black, the improvements I developed for the mathematical framework to model the kinematics of monomer (m); short polymer (p); long polymer chains (q); and inorganic nanoparticles (z) during holographic recording (Fig. 2).

$$\frac{\partial b}{\partial t} = 0, \quad (1a)$$

$$\frac{\partial m}{\partial t} + \nabla \cdot \vec{J}_m = -\Phi(t)F(x, y, t)m, \quad (1b)$$

$$\frac{\partial p}{\partial t} + \nabla \cdot \vec{J}_p = \Phi(t)F(x, y, t)m - \Phi(t)\Gamma p^2, \quad (1c)$$

$$\frac{\partial q}{\partial t} = \Phi(t)\Gamma p^2, \quad (1d)$$

$$\frac{\partial z}{\partial t} + \nabla \cdot \vec{J}_z = 0. \quad (1e)$$

$$F(x, y, t) = k_p \left[I_0 e^{-\zeta(T-y)} \right]^a \left\{ 1 + e^{-\xi z} \cos \left[\frac{2\pi \cos \phi_r(t)}{\Lambda(t)} x - \frac{2\pi \sin \phi_r(t)}{\Lambda(t)} y \right] \right\}, \quad (2)$$

$$\vec{J}_m = -D_m \frac{\partial m}{\partial x} \vec{i} - D_m \frac{\partial m}{\partial y} \vec{j}, \quad (3a)$$

$$\vec{J}_p = -D_p \left\{ \left[\frac{\partial p}{\partial x} \vec{i} + \frac{\partial p}{\partial y} \vec{j} \right] + \epsilon_z \left[\frac{\partial(pz)}{\partial x} \vec{i} + \frac{\partial(pz)}{\partial y} \vec{j} \right] \right\}, \quad (3b)$$

$$\vec{J}_z = -D_z \left\{ \left[\frac{\partial z}{\partial x} \vec{i} + \frac{\partial z}{\partial y} \vec{j} \right] + \epsilon_z \left[\frac{\partial(pz)}{\partial x} \vec{i} + \frac{\partial(pz)}{\partial y} \vec{j} \right] + \epsilon_z \left[\frac{\partial(qz)}{\partial x} \vec{i} + \frac{\partial(qz)}{\partial y} \vec{j} \right] \right\}. \quad (3c)$$

$$0 \leq x \leq \frac{\Lambda}{\cos \phi_r(t)}, \quad 0 \leq y \leq T(t), \quad t \geq 0. \quad (4)$$

Boundary Immobilization

$$\frac{\partial B}{\partial t} = \frac{Y}{u} \frac{du}{dt} \frac{\partial B}{\partial Y}, \quad (5a)$$

$$\frac{\partial M}{\partial t} = \frac{Y}{u} \frac{du}{dt} \frac{\partial M}{\partial Y} + \alpha_m^{(x)} \frac{\partial^2 m}{\partial x^2} + \alpha_m^{(y)} \frac{1}{u^2} \frac{\partial^2 M}{\partial Y^2} - \Phi(t) \beta F^*(x, Y, t) M, \quad (5b)$$

$$\begin{aligned} \frac{\partial P}{\partial t} = & \frac{Y}{u} \frac{du}{dt} \frac{\partial P}{\partial Y} + \alpha_p^{(x)} \frac{\partial^2 P}{\partial x^2} + \alpha_p^{(y)} \frac{1}{u^2} \frac{\partial^2 P}{\partial Y^2} + \alpha_{pz}^{(x)} \frac{\partial^2(PZ)}{\partial x^2} + \\ & \alpha_{pz}^{(y)} \frac{1}{u^2} \frac{\partial^2(PZ)}{\partial Y^2} + \Phi \beta F^*(x, Y, t) M - \Phi(t) \gamma P^2, \end{aligned} \quad (5c)$$

$$\frac{\partial Q}{\partial t} = \frac{Y}{u} \frac{du}{dt} \frac{\partial Q}{\partial Y} + \Phi(t) \gamma P^2, \quad (5d)$$

$$\begin{aligned} \frac{\partial Z}{\partial t} = & \frac{Y}{u} \frac{du}{dt} \frac{\partial Z}{\partial Y} + \alpha_z^{(x)} \frac{\partial^2 Z}{\partial x^2} + \alpha_z^{(y)} \frac{1}{u^2} \frac{\partial^2 Z}{\partial Y^2} + \alpha_{pz}^{(x)} \frac{\partial^2(PZ)}{\partial x^2} + \\ & \frac{1}{u^2} \alpha_{pz}^{(y)} \frac{\partial^2(PZ)}{\partial Y^2} + \alpha_{qz}^{(x)} \frac{\partial^2(QZ)}{\partial x^2} + \alpha_{qz}^{(y)} \frac{1}{u^2} \frac{\partial^2(QZ)}{\partial Y^2}, \end{aligned} \quad (5e)$$

$$F^*(x, Y, t) = e^{-a\zeta^* u(1-Y)} \left\{ 1 + e^{-\xi^* Z} \cos \left[2\pi \left(x - \frac{T_0}{\hat{x}} \tan \phi_r u Y \right) \right] \right\} \quad (6)$$

Initial & Boundary Conditions

$$\begin{aligned} M(x, Y, 0) &= 1, & P(x, Y, 0) &= 0, & Q(x, Y, 0) &= 0, & Z(x, Y, 0) &= 1, \\ B(x, Y, 0) &= 1, & u(0) &= 1, & u'(0) &= 0. \end{aligned} \quad (7)$$

$$\frac{\partial^n M}{\partial x^n}(0, Y, t) = \frac{\partial^n M}{\partial x^n}(1, Y, t) \quad n = \{0, 1, 2, \dots\}, \quad (8a)$$

$$\frac{\partial^n P}{\partial x^n}(0, Y, t) = \frac{\partial^n P}{\partial x^n}(1, Y, t) \quad n = \{0, 1, 2, \dots\}, \quad (8b)$$

$$\frac{\partial^n Z}{\partial x^n}(0, Y, t) = \frac{\partial^n Z}{\partial x^n}(1, Y, t) \quad n = \{0, 1, 2, \dots\}. \quad (8c)$$

$$\frac{\partial M}{\partial Y}(x, 0, t) = \frac{\partial P}{\partial Y}(x, 0, t) = \frac{\partial Q}{\partial Y}(x, 0, t) = \frac{\partial Z}{\partial Y}(x, 0, t) = \frac{\partial B}{\partial Y}(x, 0, t) = 0, \quad (8d)$$

$$\frac{\partial M}{\partial Y}(x, 1, t) = \frac{\partial P}{\partial Y}(x, 1, t) = \frac{\partial Q}{\partial Y}(x, 1, t) = \frac{\partial Z}{\partial Y}(x, 1, t) = \frac{\partial B}{\partial Y}(x, 1, t) = 0. \quad (8e)$$

Numerical Scheme

Numerical simulation can be done using the Crank-Nicolson implicit finite-difference scheme. For example Eqn. 5c would be ...

$$\begin{aligned} \frac{M_{i,j}^{k+1} - M_{i,j}^k}{\Delta t} = & \frac{Y_j}{u_k} \frac{u_k - u_{k-1}}{\Delta t} \left(\frac{M_{i,j+1}^{k+1} - M_{i,j-1}^{k+1}}{4\Delta Y} + \frac{M_{i,j+1}^k - M_{i,j-1}^k}{4\Delta Y} \right) + \\ & \frac{\alpha_{mm}}{2} \left[\frac{M_{i-1,j}^{k+1} - 2M_{i,j}^{k+1} + M_{i+1,j}^{k+1}}{\Delta x^2} + \frac{M_{i-1,j}^k - 2M_{i,j}^k + M_{i+1,j}^k}{\Delta x^2} \right] + \\ & \frac{\alpha_{mm}}{2u_k^2} \left[\frac{M_{i,j-1}^{k+1} - 2M_{i,j}^{k+1} + M_{i,j+1}^{k+1}}{\Delta Y^2} + \frac{M_{i,j-1}^k - 2M_{i,j}^k + M_{i,j+1}^k}{\Delta Y^2} \right] + \\ & - \Phi^k \beta F_{i,j}^k \left(\frac{M_{i,j}^{k+1} + M_{i,j}^k}{2} \right), \end{aligned} \quad (9a)$$

$$\begin{aligned}
\frac{P_{i,j}^{k+1} - P_{i,j}^k}{\Delta t} = & \frac{Y_j}{u_k} \frac{u_k - u_{k-1}}{\Delta t} \left(\frac{P_{i,j+1}^{k+1} - M_{i,j-1}^{k+1}}{4\Delta Y} + \frac{P_{i,j+1}^k - M_{i,j-1}^k}{4\Delta Y} \right) + \\
& \frac{\alpha_{pp}}{2} \left[\frac{P_{i-1,j}^{k+1} - 2P_{i,j}^{k+1} + P_{i+1,j}^{k+1}}{\Delta x^2} + \frac{P_{i-1,j}^k - 2P_{i,j}^k + P_{i+1,j}^k}{\Delta x^2} \right] + \\
& \frac{\alpha_{pp}}{2u_k^2} \left[\frac{P_{i,j-1}^{k+1} - 2P_{i,j}^{k+1} + P_{i,j+1}^{k+1}}{\Delta Y^2} + \frac{P_{i,j-1}^k - 2P_{i,j}^k + P_{i,j+1}^k}{\Delta Y^2} \right] + \\
& \frac{\alpha_{pz}}{2} \left[\frac{Z_{i-1,j}^k P_{i-1,j}^{k+1} - 2Z_{i,j}^k P_{i,j}^{k+1} + Z_{i+1,j}^k P_{i+1,j}^{k+1}}{\Delta x^2} + \right. \\
& \left. \frac{Z_{i-1,j}^k P_{i-1,j}^k - 2Z_{i,j}^k P_{i,j}^k + Z_{i+1,j}^k P_{i+1,j}^k}{\Delta x^2} \right] + \\
& \frac{\alpha_{pz}}{2u_k^2} \left[\frac{Z_{i,j-1}^k P_{i,j-1}^{k+1} - 2Z_{i,j}^k P_{i,j}^{k+1} + Z_{i,j+1}^k P_{i,j+1}^{k+1}}{\Delta Y^2} + \right. \\
& \left. \frac{Z_{i,j-1}^k P_{i,j-1}^k - 2Z_{i,j}^k P_{i,j}^k + Z_{i,j+1}^k P_{i,j+1}^k}{\Delta Y^2} \right] + \\
& \Phi^k \beta F_{i,j}^k \left(\frac{M_{i,j}^{k+1} + M_{i,j}^k}{2} \right) - \Phi^k \gamma P_{i,j}^k \left(\frac{P_{i,j}^k + P_{i,j}^{k+1}}{2} \right), \tag{9b}
\end{aligned}$$

$$\frac{Q_{i,j}^{k+1} - Q_{i,j}^k}{\Delta t} = \frac{Y_j}{u_k} \frac{u_k - u_{k-1}}{\Delta t} \left(\frac{Q_{i,j+1}^{k+1} - Q_{i,j-1}^{k+1}}{4\Delta Y} + \frac{Q_{i,j+1}^k - Q_{i,j-1}^k}{4\Delta Y} \right) + \Phi^k \gamma P_{i,j}^k \left(\frac{P_{i,j}^k + P_{i,j}^{k+1}}{2} \right), \tag{9c}$$

$$\begin{aligned}
\frac{Z_{i,j}^{k+1} - Z_{i,j}^k}{\Delta t} = & \frac{Y_j}{u_k} \frac{u_k - u_{k-1}}{\Delta t} \left(\frac{Z_{i,j+1}^{k+1} - Z_{i,j-1}^{k+1}}{4\Delta Y} + \frac{Z_{i,j+1}^k - Z_{i,j-1}^k}{4\Delta Y} \right) + \\
& \frac{\alpha_{zz}}{2} \left[\frac{Z_{i-1,j}^{k+1} - 2Z_{i,j}^{k+1} + Z_{i+1,j}^{k+1}}{\Delta x^2} + \frac{Z_{i-1,j}^k - 2Z_{i,j}^k + Z_{i+1,j}^k}{\Delta x^2} \right] + \\
& \frac{\alpha_{zz}}{2u_k^2} \left[\frac{Z_{i,j-1}^{k+1} - 2Z_{i,j}^{k+1} + Z_{i,j+1}^{k+1}}{\Delta Y^2} + \frac{Z_{i,j-1}^k - 2Z_{i,j}^k + Z_{i,j+1}^k}{\Delta Y^2} \right] + \\
& \frac{\alpha_{zp}}{2} \left[\frac{P_{i-1,j}^k Z_{i-1,j}^{k+1} - 2P_{i,j}^k Z_{i,j}^{k+1} + P_{i+1,j}^k Z_{i+1,j}^{k+1}}{\Delta x^2} \right] + \\
& \frac{\alpha_{zp}}{2} \left[\frac{P_{i-1,j}^k Z_{i-1,j}^k - 2P_{i,j}^k Z_{i,j}^k + P_{i+1,j}^k Z_{i+1,j}^k}{\Delta x^2} \right] + \\
& \frac{\alpha_{zp}}{2u_k^2} \left[\frac{P_{i,j-1}^k Z_{i,j-1}^{k+1} - 2P_{i,j}^k Z_{i,j}^{k+1} + P_{i,j+1}^k Z_{i,j+1}^{k+1}}{\Delta Y^2} \right] + \\
& \frac{\alpha_{zp}}{2u_k^2} \left[\frac{P_{i,j-1}^k Z_{i,j-1}^k - 2P_{i,j}^k Z_{i,j}^k + P_{i,j+1}^k Z_{i,j+1}^k}{\Delta Y^2} \right] + \\
& \frac{\alpha_{zq}}{2} \left[\frac{Q_{i-1,j}^k Z_{i-1,j}^{k+1} - 2Q_{i,j}^k Z_{i,j}^{k+1} + Q_{i+1,j}^k Z_{i+1,j}^{k+1}}{\Delta x^2} \right] + \\
& \frac{\alpha_{zq}}{2} \left[\frac{Q_{i-1,j}^k Z_{i-1,j}^k - 2Q_{i,j}^k Z_{i,j}^k + Q_{i+1,j}^k Z_{i+1,j}^k}{\Delta x^2} \right] + \\
& \frac{\alpha_{zq}}{2u_k^2} \left[\frac{Q_{i,j-1}^k Z_{i,j-1}^{k+1} - 2Q_{i,j}^k Z_{i,j}^{k+1} + Q_{i,j+1}^k Z_{i,j+1}^{k+1}}{\Delta Y^2} \right] + \\
& \frac{\alpha_{zq}}{2u_k^2} \left[\frac{Q_{i,j-1}^k Z_{i,j-1}^k - 2Q_{i,j}^k Z_{i,j}^k + Q_{i,j+1}^k Z_{i,j+1}^k}{\Delta Y^2} \right], \tag{9d}
\end{aligned}$$

$$F_{i,j}^k = \exp[-a\zeta^* u_k(1 - Y_j)] [1 + \exp(-\xi^* Z_{i,j}^k) \cos(2\pi x - 2\pi \tan \phi_r^k u_k Y_j)] . \quad (9e)$$

$$\begin{aligned} \int_0^1 \int_0^1 M \, dx \, dY \approx M_k^* = \frac{\Delta x^2}{4} [& M_{0,0}^k + M_{0,J}^k + M_{J,0}^k + M_{J,J}^k + 2M_{0,1}^k + \dots + 2M_{0,J-1}^k + \\ & 2M_{1,0}^k + \dots + 2M_{J-1,0}^k + 2M_{1,J}^k + \dots + 2M_{J-1,J}^k + 2M_{J,1}^k + \dots \\ & + 2M_{J,J-1}^k + 4M_{2,2}^k + \dots + 4M_{J-2,J-2}^k] , \end{aligned} \quad (10a)$$

$$\begin{aligned} \int_0^1 \int_0^1 P \, dx \, dY \approx P_k^* = \frac{\Delta x^2}{4} [& P_{0,0}^k + P_{0,J}^k + P_{J,0}^k + P_{J,J}^k + 2P_{0,1}^k + \dots + 2P_{0,J-1}^k + \\ & 2P_{1,0}^k + \dots + 2P_{J-1,0}^k + 2P_{1,J}^k + \dots + 2P_{J-1,J}^k + 2P_{J,1}^k + \dots \\ & + 2P_{J,J-1}^k + 4P_{2,2}^k + \dots + 4P_{J-2,J-2}^k] , \end{aligned} \quad (10b)$$

$$\begin{aligned} \int_0^1 \int_0^1 Q \, dx \, dY \approx Q_k^* = \frac{\Delta x^2}{4} [& Q_{0,0}^k + Q_{0,J}^k + Q_{J,0}^k + Q_{J,J}^k + 2Q_{0,1}^k + \dots + 2Q_{0,J-1}^k + \\ & 2Q_{1,0}^k + \dots + 2Q_{J-1,0}^k + 2Q_{1,J}^k + \dots + 2Q_{J-1,J}^k + 2Q_{J,1}^k + \dots \\ & + 2Q_{J,J-1}^k + 4Q_{2,2}^k + \dots + 4Q_{J-2,J-2}^k] , \end{aligned} \quad (10c)$$

$$u_k = \left[\frac{b_0}{\rho_b} + \frac{1}{\rho_m} + \frac{z_0}{\rho_z} \right]^{-1} \left[\frac{b_0}{\rho_b} + \frac{M_k^*}{\rho_m} + \frac{P_k^*}{\rho_p} + \frac{Q_k^*}{\rho_p} + \frac{z_0}{\rho_z} \right] , \quad (10d)$$

$$\phi_r^k = \tan^{-1} \left(\frac{\tan \phi_r^0}{u_k} \right) , \quad (10e)$$

$$\Lambda_k = \Lambda_0 \frac{\cos \phi_r^k}{\cos \phi_r^0} . \quad (10f)$$

Refractive Index Modulation

$$\frac{n^2 - 1}{n^2 + 2} = \phi_m \frac{n_m^2 - 1}{n_m^2 + 2} + \phi_p \frac{n_p^2 - 1}{n_p^2 + 2} + \phi_q \frac{n_q^2 - 1}{n_q^2 + 2} + \phi_z \frac{n_z^2 - 1}{n_z^2 + 2} + \phi_b \frac{n_b^2 - 1}{n_b^2 + 2} . \quad (11)$$

Solving the Lorentz-Lorenz equation will give the RI of the nanocomposite as a function of x and t . The nanocomposite RI, $n(x, t)$, can be represented by a Fourier expansion series

$$n(x, y, t) \approx \sum_{i=0} A_i(y, t) \cos \left(\frac{2\pi}{\Lambda} ix \right) + B_i(y, t) \sin \left(\frac{2\pi}{\Lambda} ix \right) , \quad (12)$$

$$A_0(y, t) = \frac{1}{\Lambda} \int_0^\Lambda n(x, y, t) \, dx , \quad (13a)$$

$$A_1(y, t) = \frac{2}{\Lambda} \int_0^\Lambda n(x, y, t) \cos \left(\frac{2\pi}{\Lambda} x \right) \, dx , \quad (13b)$$

$$B_1(y, t) = \frac{2}{\Lambda} \int_0^\Lambda n(x, y, t) \sin \left(\frac{2\pi}{\Lambda} x \right) \, dx . \quad (13c)$$

RI modulation can be modelled as

$$\Delta n(y, t) = 2\sqrt{A_1^2 + B_1^2} . \quad (14)$$

Shrinkage Modelling

We can calculate the volume at time t if we have expressions for the total volume of monomer, short polymer, cross-linked polymer and nanoparticles inside the grating

$$v(t) = \frac{1}{\rho_m} \left[\frac{1}{\hat{x}T(t)} \int_0^{T(t)} \int_0^{\hat{x}} m \, dx \, dy \right] + \frac{1}{\rho_p} \left[\frac{1}{\hat{x}T(t)} \int_0^{T(t)} \int_0^{\hat{x}} p \, dx \, dy \right] + \frac{1}{\rho_p} \left[\frac{1}{\hat{x}T(t)} \int_0^{T(t)} \int_0^{\hat{x}} q \, dx \, dy \right] + \frac{1}{\rho_z} \left[\frac{1}{\hat{x}T(t)} \int_0^{T(t)} \int_0^{\hat{x}} z \, dx \, dy \right] + \frac{1}{\rho_b} \left[\frac{1}{\hat{x}T(t)} \int_0^{T(t)} \int_0^{\hat{x}} b \, dx \, dy \right]. \quad (15)$$

An important assumptions of the fringe-plane rotation model is that all loss of volume due to polymerization takes place in the thickness of the recording medium

$$u(t) = \frac{T(t)}{T_0} = \frac{v(t)}{v(0)}. \quad (16)$$

$$u(t) = \left[\frac{1}{\rho_b} \frac{b_0}{m_0} + \frac{1}{\rho_m} + \frac{1}{\rho_z} \frac{z_0}{m_0} \right]^{-1} \left[\int_0^1 \int_0^1 \frac{M}{\rho_m} + \frac{P}{\rho_p} + \frac{Q}{\rho_p} + \frac{z_0/m_0 Z}{\rho_z} + \frac{b_0/m_0}{\rho_b} \, dx \, dY \right], \quad (17)$$

$$\text{Actual Shrinkage} = \frac{u(0) - u(t)}{u(0)} = 1 - u(t), \quad (18)$$

$$\phi_r(t) = \tan^{-1} \left[\frac{\tan \phi_r(0)}{u(t)} \right], \quad (19a)$$

$$\Lambda(t) = \hat{x} \cos \phi_r(t), \quad (19b)$$

$$\bar{n}(t) = \frac{1}{\hat{x}T} \int_0^T \int_0^{\hat{x}} n(x, y, t) \, dx \, dy = \int_0^1 \int_0^1 n(x, Y, t) \, dx \, dY, \quad (20)$$

$$\theta_B(t) = \sin^{-1} \left(\frac{\lambda_r}{2\bar{n}(t)\Lambda(t)} \right) - \phi_r(t), \quad (21)$$

$$\text{Apparent Shrinkage} = 1 - \frac{\tan \phi_r(0)}{\tan [\phi_r(0) + \Delta\theta_B]}. \quad (22)$$

Python Script

```

1 from warnings import filterwarnings
2 filterwarnings("ignore")
3 import numpy as np
4 import pandas as pd
5 from math import pi, factorial
6 from numpy.linalg import inv
7 from time import time as gettime
8 from simpsons_rule_1D import simpsons_rule_1D
9 from trapezoidal_rule_integration import trapezoidal_rule_integration
10
11 class HolographicGrating:
12
13     def __init__(
14         self,
15         start_exp=0, # Start of exposure
16         end_exp=1e2, # End of exposure
17         total_time=1e2, # Total simulation time
18         lpmm=1e3, # Spatial frequency
19         I0=5, # Intensity of recording beam
20         slant_angle=1e-4, # Grating slant_angle
21         xi=0.3, # Scattering coefficient

```

```

22     n_m=1.55,# Monomer refractive index
23     rhom=1.15,# Monomer density
24     Dm=1.6e-7,# Monomer diffusion coefficient
25     Dp=6.35e-10,# Polymer diffusion coefficient
26     rhop=1.3,# Polymer density
27     n_p=1.56,# Oligomer refractive index
28     n_q=1.64,# Polymer refractive index
29     Gamma=1,# Rate of immobilization
30     wt_pc=5e-2,# Doping %
31     Dz=1e-10,# Nanoparticle self-diffusion coefficient
32     epsilon_pz=13,# Cross-diffusion ratio
33     epsilon_qz=13,# Cross-diffusion ratio
34     rhoz=1.74,# Nanoparticle mass density
35     n_z=1.366,# Nanoparticle refractive index
36     b0=5.05,# Ratio of binder to monomer mass
37     n_b=1.5,# Binder refractive index
38     rhob=1.19,# Binder mass density
39     T0=50e-4,# Depth of photosensitive layer [cm]
40     zeta=139,# absorption coefficient [cm**-1]
41     lambda_probe=633e-7,# Wavelength of reconstruction beam
42     Delta_t=1/100,# Numerical scheme time step
43     Delta_x=1/20,# Numerical scheme spatial step
44     output_time_step=1# Seconds
45 ):
46
47     self.total_time = total_time
48     self.end_exp = end_exp
49     self.lpmm = lpmm
50     self.T0 = T0
51     self.I0 = I0
52     if slant_angle == 0:
53         self.slant_angle = 1e-5
54     else:
55         self.slant_angle = slant_angle
56     self.T0 = T0
57     self.zeta = zeta
58     self.xi = xi
59     self.Dm = Dm
60     self.n_m = n_m
61     self.rhom = rhom
62     self.Dp = Dp
63     self.rhop = rhop
64     self.n_p = n_p
65     self.n_q = n_q
66     self.Gamma = Gamma
67     self.Dz = Dz
68     self.epsilon_pz = epsilon_pz
69     self.epsilon_qz = epsilon_qz
70     self.wt_pc = wt_pc
71     self.rhoz = rhoz
72     self.n_z = n_z
73     self.b0 = b0
74     self.n_b = n_b
75     self.rhob = rhob
76     self.lambda_probe = lambda_probe
77     self.Delta_x = Delta_x
78     self.Nx = int(1/Delta_x) + 1
79     self.Delta_Y = Delta_x
80     self.Delta_t = Delta_t
81     self.output_time_step = output_time_step
82
83
84     start_computation = gettime()
85     # 1.2 --- Define parameters
86     Nx=int(1/self.Delta_x) + 1# Number of spatial points
87     Ny=int(1/self.Delta_Y) + 1# Number of spatial points
88     if Nx%2==0:
89         return "Number of x mesh points must be an odd number."
90     if Ny%2==0:
91         return "Number of y mesh points must be an odd number."
92
93     x=np.linspace(0,1,Nx)# Non-dimensional grating distance
94     n_iterations = int(self.total_time/self.Delta_t)+1# Total number of iterations
95     r=self.Delta_t/self.Delta_x/self.Delta_x# Ratio of finite time step to squared finite

```

```

spatial step
96     m0=1# Initial mass of monomer
97     t0=1 # Reference time [s]
98     Lambda0=1/10/self.lpmmm # Grating period [cm]
99     Lambda1=Lambda0
100     j_end_exp = self.end_exp/self.Delta_t # Iteration of exposure end
101     z0 = self.wt_pc/(1 - self.wt_pc)*(m0 + self.b0)# Initial nanoparticle to monomer
102     self.z0 = z0
103
104     # 1.3 --- Matrix initial conditions
105     u1=1
106     du_dt=0
107     m1 = np.ones(Nx*Nx)# m at j=0
108     p1 = np.zeros(Nx*Nx)# p at j=0
109     q1 = np.zeros(Nx*Nx)# q at j=0
110     z1 = np.ones(Nx*Nx)# z at j=0
111     b1 = self.b0*np.ones(Nx*Nx)# b at j=0
112
113     Volume0=m0/self.rhom + self.b0/self.rhob + z0/self.rhoz
114
115     phi_m0=m0/self.rhom/Volume0
116     phi_z0=z0/self.rhoz/Volume0
117     phi_b0=self.b0/self.rhob/Volume0
118
119     Lorentz_Lorenz_RHS = phi_m0*(self.n_m*self.n_m - 1)/(self.n_m*self.n_m + 2) + phi_b0*(self
.n_b*self.n_b - 1)/(self.n_b*self.n_b + 2) + phi_z0*(self.n_z*self.n_z - 1)/(self.n_z*self.n_z
+ 2)
120
121     Initial_RI = np.sqrt((2*Lorentz_Lorenz_RHS + 1)/(1 - Lorentz_Lorenz_RHS))
122
123     inside_ri = Initial_RI
124
125     n1=Initial_RI*np.ones(Nx*Nx)
126
127     phi_0=np.arcsin(np.sin(self.slant_angle/180*pi)/Initial_RI)
128     phi_1=phi_0
129     theta_B0=np.arcsin(self.lambda_probe/2/Initial_RI/Lambda0) - phi_0
130     y_hat0=Lambda0/np.sin(phi_0)
131     y_hat1=y_hat0
132     x_hat=Lambda0/np.cos(phi_0)
133
134     # 1.5 --- Nondimensionalized parameters
135     alpha_m_x=self.Dm*t0/x_hat/x_hat
136     alpha_m_y=self.Dm*t0/self.T0/self.T0
137     alpha_p_x=self.Dp*t0/x_hat/x_hat
138     alpha_p_y=self.Dp*t0/self.T0/self.T0
139     alpha_z_x=self.Dz*t0/x_hat/x_hat
140     alpha_z_y=self.Dz*t0/self.T0/self.T0
141     F0=0.1*self.I0**0.3
142     beta=F0*t0
143     gamma = self.Gamma*m0*t0
144     zeta_star = self.zeta*self.T0
145     xi_star=self.xi*z0
146     interior_points = list(range(1,Nx-1))
147     times_4 = [interior_points[i] for i in range(len(interior_points)) if interior_points[i]%2
!= 0]
148     times_2 = [interior_points[i] for i in range(len(interior_points)) if interior_points[i]%2
== 0]
149     Y=np.arange(0,1+self.Delta_x, self.Delta_x)
150     time=np.arange(1,self.total_time+self.output_time_step, self.output_time_step)
151     Y1=[]
152     x1=[]
153     for i in range(Nx):
154         for j in list(Y):
155             Y1.append(j)
156         for j in x:
157             x1.append(j)
158     Y1=np.sort(Y1)
159     indexDF=pd.DataFrame({'x':x1, 'y':Y1})
160
161     spatial_profile_DF=pd.DataFrame({'x': x1, 'Y': Y1, "monomer": m1, "short_polymer": p1, "
immobile_polymer": q1, 'nanoparticles': z1, 'binder':b1, 'refractive_index': n1, "time": np.
zeros(len(n1))})
162

```

```

163     optical_properties_DF=pd.DataFrame({'Y': Y, "time": np.zeros(len(Y)), 'N0':np.zeros(len(Y))
164 )+Initial_RI, 'Delta_n':np.zeros(len(Y)), 'nu':np.zeros(len(Y)), 'd2':np.zeros(len(Y))})
165
166     shrinkage_DF=pd.DataFrame({'time':[0], 'phi_t':[phi_0], 'Lambda_t':[Lambda0], 'theta_B':[
167 theta_B0], 'actual_shrinkage':[0], 'apparent_shrinkage':[0], 'Thickness':[self.T0], 'Mean_RI':[
168 Initial_RI]})
169
170
171     def simpsons_rule_1D(arr1D):
172
173         intpts=list(range(1,len(arr1D)-1))
174         times_4=[i for i in intpts if i%2!=0]
175         times_2=[i for i in intpts if i%2==0]
176
177         return self.Delta_x/3*(arr1D[0] + 4*sum(arr1D[times_4]) + 2*sum(arr1D[times_2]) +
178 arr1D[len(arr1D)-1])
179
180
181     # 1.6 --- Calculate each time step via implicit finite difference method
182     for j in range(1,n_iterations):
183
184         if j <= j_end_exp:
185             Phi=1
186         else:
187             Phi = 0# Phi=1 if illumination is on, 0 otherwise
188
189         f = np.zeros(Nx*Nx).reshape(Nx,Nx)
190         for i in range(Nx):
191             matrix_z1=z1.reshape(Nx,Nx)
192             z1_i=matrix_z1[:,i]
193             f[:,i] = np.exp(-0.3*zeta_star*u1*(1-Y[i]))*(1 + np.exp(-xi_star*z1_i)*np.cos(2*pi
194 *x - 2*pi*self.T0/y_hat1*u1*Y[i]))
195
196         f = f.reshape(Nx*Nx,)
197
198         MM2 = (2 + Phi*self.Delta_t*beta*f)*np.identity(Nx*Nx)
199         MM1 = (2 - Phi*self.Delta_t*beta*f)*np.identity(Nx*Nx)
200         PP2 = (2 + self.Delta_t*gamma*p1)*np.identity(Nx*Nx)
201         PP1 = (2 - self.Delta_t*gamma*p1)*np.identity(Nx*Nx)
202         PM2 = (+Phi*self.Delta_t*beta*f)*np.identity(Nx*Nx)
203         PM1 = (+Phi*self.Delta_t*beta*f)*np.identity(Nx*Nx)
204         QQ2 = 2*np.identity(Nx*Nx)
205         QQ1 = 2*np.identity(Nx*Nx)
206         QP2 = (+Phi*gamma*self.Delta_t*p1)*np.identity(Nx*Nx)
207         QP1 = (+Phi*gamma*self.Delta_t*p1)*np.identity(Nx*Nx)
208         ZZ2 = 2*np.identity(Nx*Nx)
209         ZZ1 = 2*np.identity(Nx*Nx)
210         BB2 = 2*np.identity(Nx*Nx)
211         BB1 = 2*np.identity(Nx*Nx)
212
213         for i in range(Nx*Nx):
214
215             if i in list(indexDF.loc[indexDF.x==0,].index):
216                 i_minus_1 = i+Nx-2
217             else:
218                 i_minus_1 = i-1
219
220             if i in list(indexDF.loc[indexDF.x==1,].index):
221                 i_plus_1 = i-Nx+2
222             else:
223                 i_plus_1 = i+1
224
225             if i in list(indexDF.loc[indexDF.y==0,].index):
226                 j_minus_1 = i+Nx
227             else:
228                 j_minus_1 = i-Nx
229
230             if i in list(indexDF.loc[indexDF.y==1,].index):
231                 j_plus_1 = i-Nx
232             else:
233                 j_plus_1 = i+Nx

```

```

232     MM2[i, i_minus_1] = MM2[i, i_minus_1] - r*alpha_m_x
233     MM2[i, j_minus_1] = MM2[i, j_minus_1] - r*alpha_m_y/u1/u1
234     MM2[i, i] = MM2[i, i] + 2*r*alpha_m_x
235     MM2[i, i] = MM2[i, i] + 2*r*alpha_m_y/u1/u1
236     MM2[i, i_plus_1] = MM2[i, i_plus_1] - r*alpha_m_x
237     MM2[i, j_plus_1] = MM2[i, j_plus_1] - r*alpha_m_y/u1/u1
238
239     MM1[i, i_minus_1] = MM1[i, i_minus_1] + r*alpha_m_x
240     MM1[i, j_minus_1] = MM1[i, j_minus_1] + r*alpha_m_y/u1/u1
241     MM1[i, i] = MM1[i, i] - 2*r*alpha_m_x
242     MM1[i, i] = MM1[i, i] - 2*r*alpha_m_y/u1/u1
243     MM1[i, i_plus_1] = MM1[i, i_plus_1] + r*alpha_m_x
244     MM1[i, j_plus_1] = MM1[i, j_plus_1] + r*alpha_m_y/u1/u1
245
246     PP2[i, i_minus_1] = PP2[i, i_minus_1] - r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[
i_minus_1])
247     PP2[i, j_minus_1] = PP2[i, j_minus_1] - r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*
z1[j_minus_1])
248     PP2[i, i] = PP2[i, i] + 2*r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[i])
249     PP2[i, i] = PP2[i, i] + 2*r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*z1[i])
250     PP2[i, i_plus_1] = PP2[i, i_plus_1] - r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[
i_plus_1])
251     PP2[i, j_plus_1] = PP2[i, j_plus_1] - r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*
z1[j_plus_1])
252
253     PP1[i, i_minus_1] = PP1[i, i_minus_1] + r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[
i_minus_1])
254     PP1[i, j_minus_1] = PP1[i, j_minus_1] + r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*
z1[j_minus_1])
255     PP1[i, i] = PP1[i, i] - 2*r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[i])
256     PP1[i, i] = PP1[i, i] - 2*r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*z1[i])
257     PP1[i, i_plus_1] = PP1[i, i_plus_1] + r*alpha_p_x*(1 + self.epsilon_pz*z0*z1[
i_plus_1])
258     PP1[i, j_plus_1] = PP1[i, j_plus_1] + r*alpha_p_y/u1/u1*(1 + self.epsilon_pz*z0*
z1[j_plus_1])
259
260     ZZ2[i, i_minus_1] = ZZ2[i, i_minus_1] - r*alpha_z_x*(1 + self.epsilon_qz*q1[
i_minus_1] + self.epsilon_pz*p1[i_minus_1])
261     ZZ2[i, j_minus_1] = ZZ2[i, j_minus_1] - r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[
j_minus_1] + self.epsilon_pz*p1[j_minus_1])
262     ZZ2[i, i] = ZZ2[i, i] + 2*r*alpha_z_x*(1 + self.epsilon_qz*q1[i] + self.epsilon_pz
*p1[i])
263     ZZ2[i, i] = ZZ2[i, i] + 2*r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[i] + self.
epsilon_pz*p1[i])
264     ZZ2[i, i_plus_1] = ZZ2[i, i_plus_1] - r*alpha_z_x*(1 + self.epsilon_qz*q1[i_plus_1
] + self.epsilon_pz*p1[i_plus_1])
265     ZZ2[i, j_plus_1] = ZZ2[i, j_plus_1] - r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[
j_plus_1] + self.epsilon_pz*p1[j_plus_1])
266
267     ZZ1[i, i_minus_1] = ZZ1[i, i_minus_1] + r*alpha_z_x*(1 + self.epsilon_qz*q1[
i_minus_1] + self.epsilon_pz*p1[i_minus_1])
268     ZZ1[i, j_minus_1] = ZZ1[i, j_minus_1] + r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[
j_minus_1] + self.epsilon_pz*p1[j_minus_1])
269     ZZ1[i, i] = ZZ1[i, i] - 2*r*alpha_z_x*(1 + self.epsilon_qz*q1[i] + self.epsilon_pz
*p1[i])
270     ZZ1[i, i] = ZZ1[i, i] - 2*r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[i] + self.
epsilon_pz*p1[i])
271     ZZ1[i, i_plus_1] = ZZ1[i, i_plus_1] + r*alpha_z_x*(1 + self.epsilon_qz*q1[i_plus_1
] + self.epsilon_pz*p1[i_plus_1])
272     ZZ1[i, j_plus_1] = ZZ1[i, j_plus_1] + r*alpha_z_y/u1/u1*(1 + self.epsilon_qz*q1[
j_plus_1] + self.epsilon_pz*p1[j_plus_1])
273
274     MM2[i, j_minus_1] = MM2[i, j_minus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
275     MM1[i, j_minus_1] = MM1[i, j_minus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
276     MM2[i, j_plus_1] = MM2[i, j_plus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
277     MM1[i, j_plus_1] = MM1[i, j_plus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
278
279     PP2[i, j_minus_1] = PP2[i, j_minus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
280     PP1[i, j_minus_1] = PP1[i, j_minus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
281     PP2[i, j_plus_1] = PP2[i, j_plus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
282     PP1[i, j_plus_1] = PP1[i, j_plus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
283
284     QQ2[i, j_minus_1] = QQ2[i, j_minus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
285     QQ1[i, j_minus_1] = QQ1[i, j_minus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt

```

```

286         QQ2[i, j_plus_1] = QQ2[i, j_plus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
287         QQ1[i, j_plus_1] = QQ1[i, j_plus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
288
289         ZZ2[i, j_minus_1] = ZZ2[i, j_minus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
290         ZZ1[i, j_minus_1] = ZZ1[i, j_minus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
291         ZZ2[i, j_plus_1] = ZZ2[i, j_plus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
292         ZZ1[i, j_plus_1] = ZZ1[i, j_plus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
293
294         BB2[i, j_minus_1] = BB2[i, j_minus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
295         BB1[i, j_minus_1] = BB1[i, j_minus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
296         BB2[i, j_plus_1] = BB2[i, j_plus_1] - Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
297         BB1[i, j_plus_1] = BB1[i, j_plus_1] + Y1[i]/u1*self.Delta_t/2/self.Delta_x*du_dt
298
299         m2 = np.matmul(inv(MM2), np.matmul(MM1,m1))
300
301         p2 = np.matmul(inv(PP2), np.matmul(PP1,p1) + np.matmul(PM2,m2) + np.matmul(PM1,m1))
302
303         q2 = np.matmul(inv(QQ2), np.matmul(QQ1, q1) + np.matmul(QP2,p2) + np.matmul(QP1,p1))
304
305         if z0==0:
306             z2 = z1
307         else:
308             z2 = np.matmul(inv(ZZ2), np.matmul(ZZ1,z1))
309
310         b2 = np.matmul(inv(BB2), np.matmul(BB1,b1))
311
312         Vb = b2/self.rhob # cm**3
313         Vm = m2*m0/self.rhom # cm**3
314         Vp = p2*m0/self.rhop # cm**3
315         Vq = q2*m0/self.rhop # cm**3
316         Vz = z2*z0/self.rhoz # cm**3
317         Vtotal=Vb+Vm+Vp+Vq+Vz # cm**3
318         phi_m = Vm/Vtotal
319         phi_b = Vb/Vtotal
320         phi_p = Vp/Vtotal
321         phi_q = Vq/Vtotal
322         phi_z = Vz/Vtotal
323
324         Lorentz_Lorenz_RHS = phi_m*(self.n_m*self.n_m - 1)/(self.n_m*self.n_m + 2) + phi_b*(
self.n_b*self.n_b - 1)/(self.n_b*self.n_b + 2) + phi_p*(self.n_p*self.n_p - 1)/(self.n_p*self.
n_p + 2) + phi_q*(self.n_q*self.n_q - 1)/(self.n_q*self.n_q + 2) + phi_z*(self.n_z*self.n_z -
1)/(self.n_z*self.n_z + 2)
325
326         n2=np.sqrt((2*Lorentz_Lorenz_RHS + 1)/(1 - Lorentz_Lorenz_RHS))
327         n2=n2.reshape(Nx,Nx).T
328
329         inside_ri = simpsons_rule_1D(n2[int((Nx-1)/2)])
330
331         N0=[(self.Delta_x/3*(n2[0,i] + np.sum(2*n2[times_2,i]) + np.sum(4*n2[times_4,i]) + n2
[(Nx-1),i])) for i in range(Nx)]
332
333         n2_cos=np.zeros(n2.shape)
334         for i in range(Nx):
335             n2_cos[:,i] = n2[:,i]*np.cos(2*pi*x)
336
337         n2_sin=np.zeros(n2.shape)
338         for i in range(Nx):
339             n2_sin[:,i] = n2[:,i]*np.sin(2*pi*x)
340
341         N1_a=np.array([2*self.Delta_x/3*(n2_cos[0,i] + np.sum(2*n2_cos[times_2,i]) + np.sum(4*
n2_cos[times_4,i]) + n2_cos[(Nx-1),i]) for i in range(Nx)])
342
343         N1_b=np.array([2*self.Delta_x/3*(n2_sin[0,i] + np.sum(2*n2_sin[times_2,i]) + np.sum(4*
n2_sin[times_4,i]) + n2_sin[(Nx-1),i]) for i in range(Nx)])
344
345         n_tilde=np.ones((Nx,Nx))
346         for i in range(Nx):
347             n_tilde[:,i]=N0[i]*n_tilde[:,i] + N1_a[i]*np.cos(2*pi*x) + N1_b[i]*np.sin(2*pi*x)
348
349         sq_diff=(n2 - n_tilde)**2
350
351         d2=[(self.Delta_x/3*(sq_diff[0,i] + np.sum(2*sq_diff[times_2,i]) + np.sum(4*sq_diff[
times_4,i]) + sq_diff[(Nx-1),i])) for i in range(Nx)]
352

```

```

353         n2=n2.T.reshape(Nx*Nx,)
354
355         Volume1=(trapezoidal_rule_integration(m2, self.Delta_x)/self.rhom +
trapezoidal_rule_integration(p2, self.Delta_x)/self.rhop + trapezoidal_rule_integration(q2,
self.Delta_x)/self.rhop + trapezoidal_rule_integration(z2, self.Delta_x)*z0/self.rhoz +
trapezoidal_rule_integration(b2, self.Delta_x)/self.rhob)
356
357         u1 = Volume1/Volume0
358
359         phi_1=np.arctan(np.tan(phi_0)/u1)
360
361         Lambda1=np.cos(phi_1)/np.cos(phi_0)*Lambda0
362
363         y_hat1=Lambda1/np.sin(phi_1)
364
365         theta_B=np.arcsin(self.lambda_probe/2/inside_ri/Lambda1)-phi_1
366
367         Delta_theta_B=theta_B0-theta_B,
368
369         Delta_n=np.sqrt(N1_a*N1_a+N1_b*N1_b)
370
371         nu=pi*Delta_n*self.T0*u1/self.lambda_probe/np.cos(theta_B)
372
373         m1 = m2
374         p1 = p2
375         q1 = q2
376         z1 = z2
377         b1 = b2
378         n1 = n2
379
380         if self.Delta_t*j in time:
381
382             spatial_profile_DF=pd.concat([spatial_profile_DF, pd.DataFrame({"x": x1,
'Y': Y1,"monomer": m1,"short_polymer": p1,"
immobile_polymer": q1,'nanoparticles': z1,'binder':b1,'refractive_index': n1,"time":j*self.
Delta_t*np.ones(len(n1))})]).reset_index(drop=True)
383
384             optical_properties_DF=pd.concat([optical_properties_DF,pd.DataFrame({'time':j*self.
Delta_t,'Y':Y,'N0':N0,'Delta_n':Delta_n,'d2':d2, 'nu':nu})]).reset_index(drop=True)
385
386             shrinkage_DF=pd.concat([shrinkage_DF, pd.DataFrame({'time':[self.Delta_t*j], '
Lambda_t':[Lambda1], 'phi_t':[phi_1], 'theta_B':[theta_B], 'actual_shrinkage':[1-u1], '
apparent_shrinkage':[1 - np.tan(phi_0)/np.tan(phi_0 + Delta_theta_B)][0], 'Thickness':[self.T0
*u1], 'Mean_RI':[np.mean(n1)] })]).reset_index(drop=True)
387
388
389             #Moharam_Young=lambda_probe*lambda_probe/Mean_RI/Delta_n/Lambda1/Lambda1/np.cos(phi_1)
390
391             self.Klein_Cook=2*pi*self.lambda_probe*self.T0*u1/inside_ri/Lambda1/Lambda1/np.cos(phi_1)
392
393             if self.Klein_Cook < 10:
394                 self.Geometry='Planar'
395                 J0=optical_properties_DF.nu/2
396                 J1=J0
397                 for l in range(1,101):
398                     J1 = J1 + ((-1)**l)/factorial(l)/factorial(l+1)*(J0**(2*l + 1))
399                 eta=J1*J1
400             else:
401                 self.Geometry='Volume'
402                 eta=np.sin(np.sqrt(optical_properties_DF.nu*optical_properties_DF.nu))*2
403
404             optical_properties_DF['eta']=eta
405
406             end_computation = gettime()
407
408             self.computation_duration = end_computation-start_computation
409
410             self.spatial_profile_DF = spatial_profile_DF
411
412             self.optical_properties_DF = optical_properties_DF
413
414             self.shrinkage_DF = shrinkage_DF
415
416             return

```

```

417
418 if __name__ == "__main__":
419
420     a = HolographicGrating(total_time=0)
421     assert len(a.shrinkage_DF) == 1
422     assert len(a.optical_properties_DF) == a.Nx
423     assert len(a.spatial_profile_DF) == a.Nx*a.Nx
424     del a
425     print("All tests passed\n")

```

Results

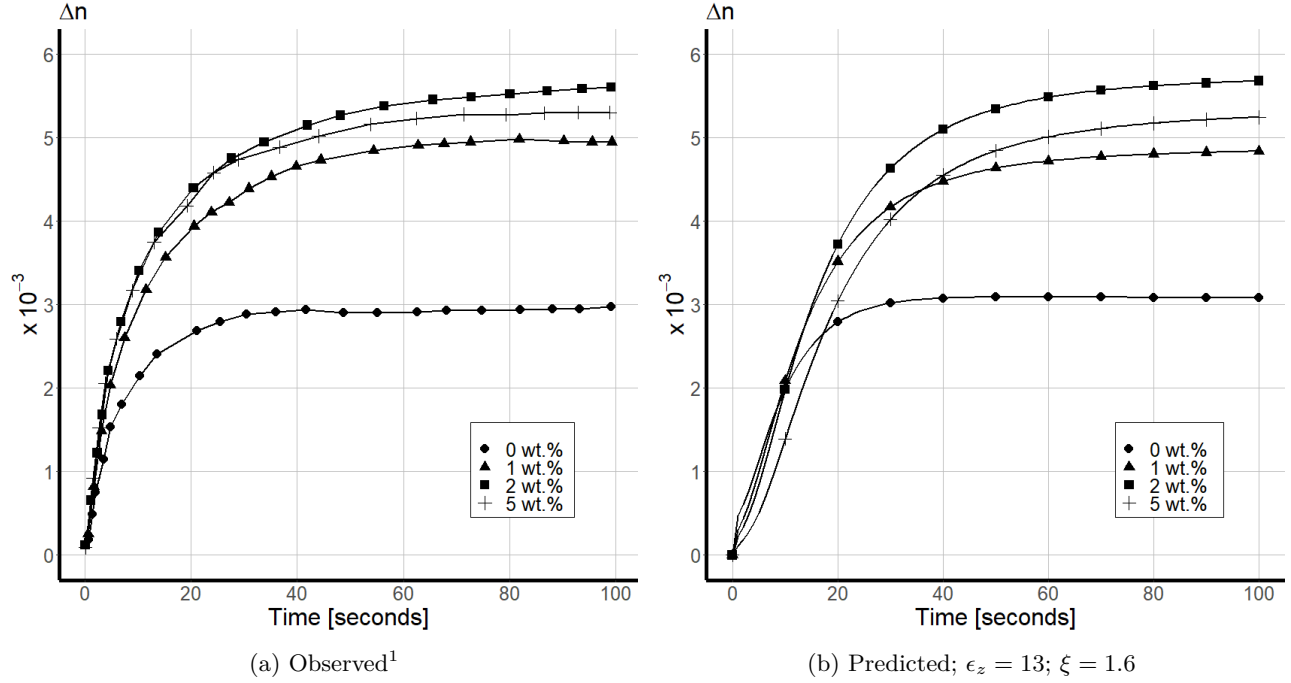
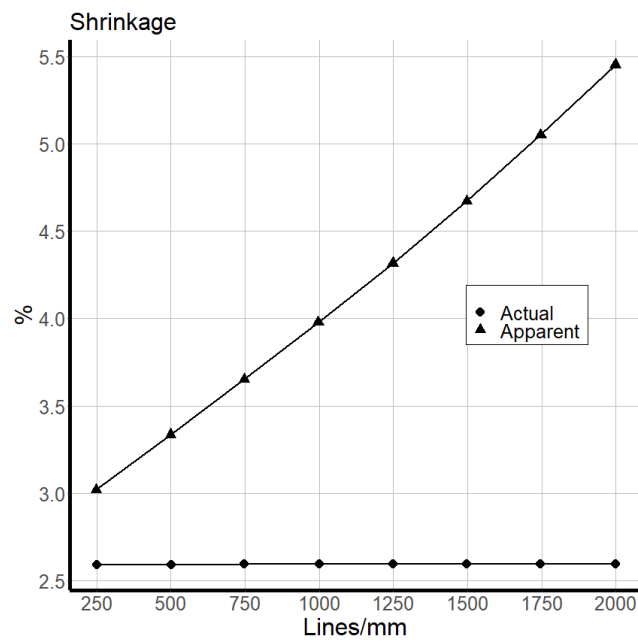
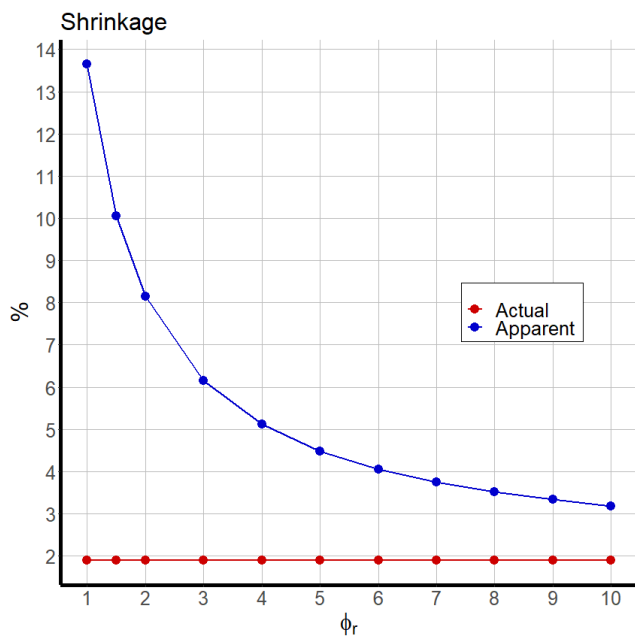


Figure 3: Time evolution of Δn of an unslanted holographic grating recorded in AA/PVA photopolymer with increased doping of BEA nanozeolites.

¹Cody, D et al. (2014), *Effect of zeolite nanoparticles on the optical properties of diacetone acrylamide-based photopolymer*, Optical Materials 37, 181-187



(a) ... spatial frequencies ranging from 250 - 2000 lines/mm



(b) ... slant angles up to 10°

Figure 4: The predicted actual and apparent shrinkage for ...